

Claude Code Skills for Academic Researchers

A Practical Introduction

Alessandro Spina

UTS Finance Department Brownbag

March 2026

Roadmap

1. The Problem
2. What Are Skills?
3. How Skills Work
4. Example Skills for Researchers
5. Setting Up Your Skills
6. Practical Advice
7. Summary

1

The Problem

The RA Who Forgets Everything

Every time you start a new Claude Code session, you are working with a **brand-new research assistant** who knows nothing about your conventions.

You end up repeating the same instructions... every single time.

- ▶ "Use `data.table`, not tidyverse"
- ▶ "Compile with `pdfLaTeX`, not XeLaTeX"
- ▶ "Always `cluster` standard errors"
- ▶ "Don't run code on licensed data without asking"

Key Insight

Skills encode this knowledge **once** — reusable across sessions and projects.

2

What Are Skills?

Skills = An Onboarding Guide for Claude

A Skill is a folder containing instructions, templates, and scripts that Claude Code loads **on-demand** when they are relevant to your task.

Think of it as writing an onboarding guide for a new RA — except the RA is an AI agent, and the guide is a Markdown file with some YAML metadata at the top.

Key property: Skills are **reusable across sessions and projects**.

Write them once, use them everywhere.

Skills vs. CLAUDE.md

Both persist instructions across sessions, but they serve different purposes:

	CLAUDE.md	Skills
Scope	Short conventions	Rich workflows
Loading	Always loaded	On-demand
Files	Single file	Folder + subfiles
Example	"Use data.table"	Full patterns + templates

Rule of Thumb

If it is a **few lines** of convention, put it in CLAUDE.md.

If it is a **full workflow** with examples, templates, or helper scripts, make it a Skill.

3

How Skills Work

Progressive Disclosure: Claude Loads What It Needs



You can install **many skills** without bloating the context window.

Anatomy of a Skill

Every Skill is a folder with one required file and optional subfolders:

<code>my-skill/</code>	
<code>SKILL.md</code>	Required
<code>scripts/</code>	Code Claude can execute
<code>references/</code>	Docs for guidance
<code>assets/</code>	Templates, themes

References

Things Claude **reads** to inform its thinking. E.g., style guides, fixest patterns.

Assets

Things Claude **copies or uses directly** in output. E.g., a Beamer preamble, a .sty file.

The SKILL.md File

Two parts: **YAML frontmatter** (metadata) and **Markdown body** (instructions).

```
---  
  
name: r-empirical-finance  
description: >  
Conventions for writing empirical finance R code  
with data.table, fixest, arrow, and ggplot2. Use  
whenever writing or reviewing R scripts for panel  
data, DiD, IV, or regressions.  
  
---  
  
# R Empirical Finance Code Standards  
  
## Package stack  
  
Always prefer data.table (never dplyr/tidyverse)...
```

The Description Is Everything

Claude uses the description field to decide whether to load a skill. It tends to **under-trigger**, so be explicit and slightly pushy.

Weak Description

"Helps with R code for research."

Strong Description

*"Conventions for writing empirical finance R code with **data.table**, **fixest**, **arrow**, and **ggplot2**. Use this skill whenever writing, reviewing, or debugging R scripts for panel data, event studies, DiD, IV/2SLS... **even if** the user doesn't explicitly mention these packages."*

Tips

- ▶ Include **trigger phrases**: "Use this skill whenever..."
- ▶ Name **specific packages** and methods
- ▶ Add "**even if** the user doesn't mention X" language

4

Example Skills for Researchers

Skill 1: R Empirical Code Standards

Encodes your preferences so Claude **always** writes R code your way:

- 1 `data.table` over tidyverse, always
- 2 `fixest::feols()` for regressions
- 3 `arrow` for Parquet I/O
- 4 Clustered SEs by default
- 5 Auto-warn: missing clustering, duplicates, bad controls

```
references/
```

```
fixest-patterns.md
```

```
Advanced patterns loaded  
only when needed:
```

- Stepwise estimation
- Split samples
- Lag operators
- IV first-stage diagnostics

Skill 2: Beamer Slide Generation

Captures your slide conventions so Claude produces **properly formatted** decks:

- 1 pdfLaTeX only (no fontspec)
- 2 UTS house style with standard preamble
- 3 One idea per slide, informative titles
- 4 Discussion Qs tagged with Bloom's levels
- 5 Case studies: Context → Tension → Question

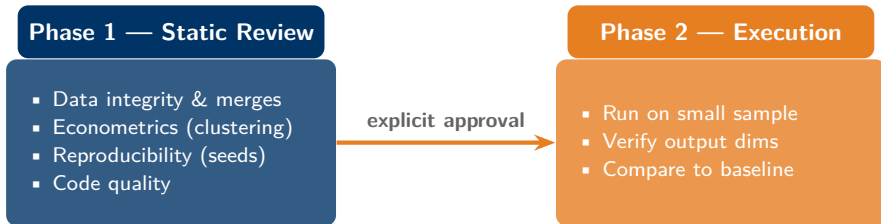
Assets Folder

The `assets/` folder holds the reusable **Beamer preamble template**.

Claude copies it directly into new decks — no need to repeat style definitions across projects.

Skill 3: Research Code Audit

A two-phase protocol for reviewing empirical research code:



The read-then-execute separation **protects licensed data** (WRDS, CRSP).

Skill 4: Structured Paper Review

When you drop a paper PDF into Claude, get a **consistent summary**:

- 1 One-sentence summary
- 2 Research question & why it matters
- 3 **Identification strategy** — method, FEs, clustering
- 4 Key findings with **actual magnitudes**
- 5 Robustness, threats, connections to your work

Key Differentiator

The **identification strategy** section is what sets this apart from a generic summary.

Forces Claude to report: exogenous variation, instrument validity, parallel trends — not just “they ran a regression.”

5

Setting Up Your Skills

Where Skills Live

Skills can be installed at different levels depending on how broadly you want them to apply:

Location	Scope	Use case
~/.claude/skills/	All projects	Personal conventions
.claude/skills/	One project	Paper-specific
Claude.ai Settings	Web UI	Upload as zip

Recommendation

Use **global** for cross-project conventions (R style, Beamer, code audit).

Use **project-level** for paper-specific workflows and data-handling rules.

Recommended Repo Structure

Host your skills in a **GitHub repo**. Clone into `~/.claude/skills/`.

```
claude-skills/  
  r-empirical-finance/  
    SKILL.md  
  references/fixest-patterns.md  
  beamer-slides/  
    SKILL.md  
  assets/uts-beamer-preamble.tex  
  code-audit/ SKILL.md  
  paper-review/ SKILL.md
```

Structure

Each skill is a **self-contained folder**.

SKILL.md is required.

Subfolders are optional.

Supporting files loaded on demand.

Getting Started: The Commands

One-time setup:

```
$ git clone  
https://github.com/you/claude-skills.git \  
"$HOME/.claude/skills"
```

Day-to-day updates:

```
$ cd "$HOME/.claude/skills"  
$ git pull
```

Important

Do **not** put this repo in Dropbox.

Git and Dropbox conflict on the internal `.git` folder.

GitHub **is** your cloud backup.

6

Practical Advice

What Makes a Good Skill

Do

- ▶ Start small — one page, one pain point
- ▶ Use **concrete examples**, not abstract rules
- ▶ Encode your corrections: 3+ repeats $\Rightarrow$ skill
- ▶ Be explicit about what *not* to do

Don't

- ▶ Encode entire methodology in one skill
- ▶ Write a “meta-skill” before 3–4 regular skills
- ▶ Test with “use skill X to...” — use natural prompts

If you have corrected Claude **3+ times** on the same thing, it belongs in a skill.

Know What Skills Cannot Do

Skills are **reactive**, not scheduled. They fire when Claude decides the skill is relevant.

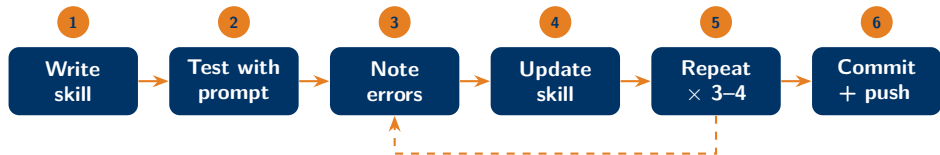
Cannot

- ▶ **Run on a cron job** (“check SSRN every Monday”)
- ▶ **Scrape websites** in the background
- ▶ **Maintain state** between sessions

Can

- ▶ **Standardise** how Claude analyses a paper
- ▶ **Enforce** coding conventions across projects
- ▶ **Implement** multi-step review protocols
- ▶ **Generate** documents from templates

The Iteration Workflow



Skills are **living documents**. Every time you correct Claude on something that should have been in the skill, go back and add it.

7

Summary

Key Takeaways

- 1 Skills **encode research conventions** so Claude remembers them across sessions and projects
- 2 They are just **folders with a SKILL.md** file — no special tooling required
- 3 Start with **2–3 skills** that address your biggest pain points
- 4 Host in a **GitHub repo**, clone into `~/.claude/skills/`
- 5 Write **pushy descriptions** so Claude actually triggers them
- 6 **Iterate** based on real usage — add corrections as you go

Resources

- ▶ **Anthropic Skills documentation**

<https://code.claude.com/docs/en/skills>

- ▶ **How to create custom Skills**

<https://support.claude.com/en/articles/12512198>

- ▶ **Anthropic engineering blog** — “Equipping agents for the real world with Agent Skills”

<https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>

- ▶ **Public skills repository**

<https://github.com/anthropics/skills>

- ▶ **Community skills list**

<https://github.com/travisvn/awesome-claude-skills>